DOCUMENTO TÉCNICO · CRIPTOGRAFÍA Y SISTEMAS DISTRIBUIDOS

La prueba de trabajo y el problema del doble gasto

Origen, matemáticas y simulaciones
de la prueba de trabajo de Bitcoin

Hash del bloque génesis (bloque 0), calculado con doble SHA-256 sobre 80 bytes:

000000000019d6689c085ae165831e934
ff763ae46a2a6c172b3f1b60a8ce26f

Diez ceros hexadecimales: 43 bits iniciales en cero. Ese es el «trabajo» que se demuestra.

Documentación base: http://math.com.mx/pt.html
Cifras de red: actualizadas a septiembre de 2026
Fecha de este documento: 13 de octubre de 2026
Gráficas y datos: generados a partir de archivos CSV reproducibles (Apéndice B)

Resumen

Antes de 2008, el dinero digital tenía un defecto de fábrica: un archivo se puede copiar sin costo, de modo que una moneda que es solo un archivo puede gastarse dos veces. La solución clásica era un banco que llevara el libro de cuentas; el reto real era evitar el doble gasto *sin* esa autoridad central. Este documento desarrolla, de forma autocontenida, el tema de la página «La prueba de trabajo y el problema del doble gasto»: las piezas criptográficas que llegaron antes de 2008, la idea de usar la prueba de trabajo como mecanismo de consenso, el funcionamiento de las funciones hash y del encabezado de bloque de 80 bytes (con verificación independiente del bloque génesis), la minería como proceso de ensayos independientes, el ajuste de dificultad y la carrera de un atacante contra la red honesta. Se compara la aproximación de Poisson de 2008 con la fórmula exacta (binomial negativa) y se contrasta con simulaciones de Monte Carlo. Se cierra con el costo energético de la solución y con las posturas que genera. Las simulaciones interactivas de la página original se sustituyen por cálculos reproducibles y por gráficas leídas de archivos de datos, que también sirven como especificación para reconstruir las visualizaciones en una página web.

Palabras clave: prueba de trabajo, doble gasto, Bitcoin, SHA-256, minería, ajuste de dificultad, binomial negativa, consenso.

Índice general

1 El problema: el doble gasto **1**

2 Antecedentes: las piezas que llegaron antes **2**

3 La idea de 2008: una CPU, un voto **2**

4 La herramienta: qué es un hash **4**

4.1 El efecto avalancha, en números 4

5 Anatomía de un bloque real **5**

5.1 El bloque génesis, campo por campo 6

5.2 El objetivo y el campo «bits» 6

5.3 La raíz de Merkle 7

6 El rompecabezas: minar un bloque **7**

6.1 Cómo calcula el hardware: el *midstate* 8

6.2 Un experimento con minería real 8

6.3 La escala: de tu navegador a la red de Bitcoin 8

7 Por qué encadenar: la cadena con más trabajo gana **10**

7.1 La regla de consenso 10

7.2 Dos formas de decidir quién llegó primero 11

8 El reloj: diez minutos por bloque, pase lo que pase **11**

8.1 Los tiempos entre bloques son exponenciales 11

8.2 Simulación del ajuste 12

9 Cómo se intentaría el doble gasto hoy 14

9.1 El modelo 14

9.2 Tres cálculos de la misma probabilidad 14

10 Simulación: la carrera contra la red honesta 15

10.1 El resultado empírico coincide con la fórmula exacta 16

10.2 El riesgo, confirmación por confirmación 17

10.3 ¿Cuántas confirmaciones hacen falta? 17

10.4 El ataque del 51 % 17

11 Lo que la prueba de trabajo aporta 19

12 El costo de esta solución 19

12.1 ¿Cuánto cuesta atacar? Una estimación de orden de magnitud 20

12.2 Dos posturas sobre si ese gasto vale la pena 20

13 Conclusiones 21

A Guía para convertir este documento en una página web 22

A.1 Marcado semántico 22

A.2 Correspondencia LaTeX → HTML 22

A.3 Gráficas: qué dibujar y de dónde sale cada número 22

A.4 Herramientas interactivas que la página debe incluir 23

A.5 Diseño, formato y accesibilidad 24

A.6 Vectores de prueba para validar la página 24

B Reproducibilidad 25

C Derivaciones breves 26

Glosario 26

Referencias 27

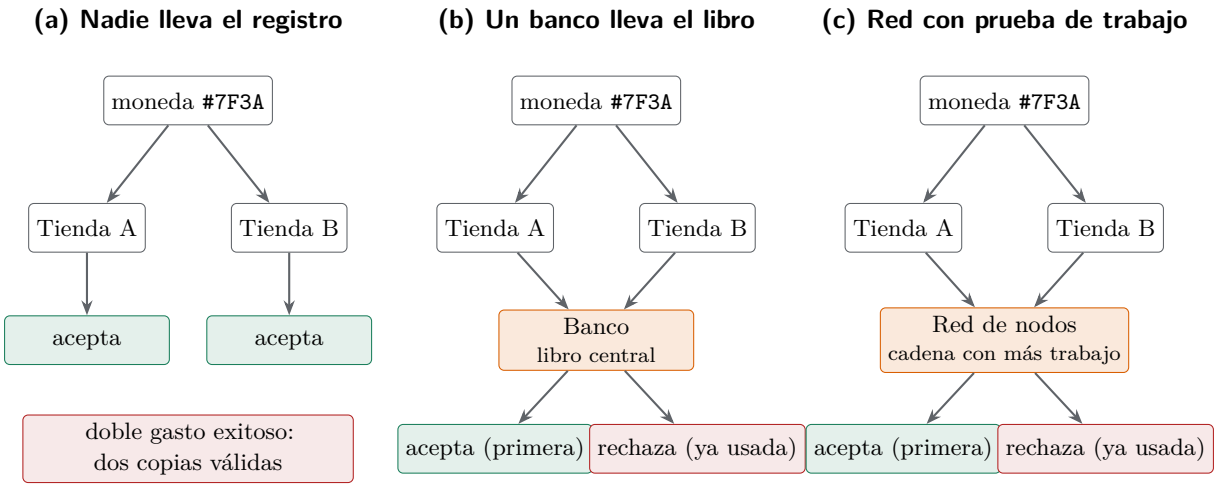


Figura 1. Quién lleva el registro. El mismo archivo-moneda se envía a dos comerciantes a la vez. Con un registro central (b) el orden lo decide el banco; con prueba de trabajo (c) lo decide la red, y cualquiera puede verificar el resultado.

1 El problema: el doble gasto

Cuando pagas con un billete físico, el problema se resuelve solo: el papel cambia de mano y ya no lo tienes. No puedes dárselo a dos personas a la vez. El dinero digital no tiene esa propiedad. Un archivo que representa «10 dólares» se puede duplicar exactamente igual que una fotografía. Incluso si la moneda lleva una firma digital, la firma prueba *quién* la emitió, no que no se haya gastado antes: dos copias de un archivo firmado son igual de válidas.

> **Definición 1.1 — Doble gasto** — Se llama *doble gasto* a la posibilidad de que un mismo activo digital se utilice en dos o más pagos distintos, porque cada pago, visto por separado, es indistinguible de un pago legítimo.

La solución tradicional siempre fue la misma: poner a alguien de confianza en el medio. Un banco, una red de tarjetas, un procesador de pagos. Esa entidad lleva un libro de cuentas central y, cuando gastas una moneda, la marca como usada. El doble gasto se evita porque hay una sola versión de la verdad, custodiada por un tercero.

El reto, entonces, no era solo «evitar el doble gasto» —eso ya se sabía hacer con un banco de por medio—. El reto real era evitarlo *sin* esa autoridad central, entre desconocidos que no tienen por qué confiar unos en otros. En palabras técnicas: lograr que una red abierta se ponga de acuerdo sobre el *orden* de las transacciones, porque, si hay orden, la primera gana y la segunda se rechaza.

> **Nota 1.1 — El orden resuelve el problema** — Supón que existe un orden total y acordado de todas las transacciones, y la regla «una moneda solo puede gastarse una vez; si dos transacciones la gastan, vale la primera en ese orden». Entonces el doble gasto es imposible por construcción: la segunda transacción es simplemente inválida. Todo el diseño de Bitcoin consiste en conseguir ese orden común sin coordinador, sin identidades confiables y frente a participantes que pueden mentir. Las secciones siguientes muestran cómo la prueba de trabajo lo hace posible y con qué garantías.

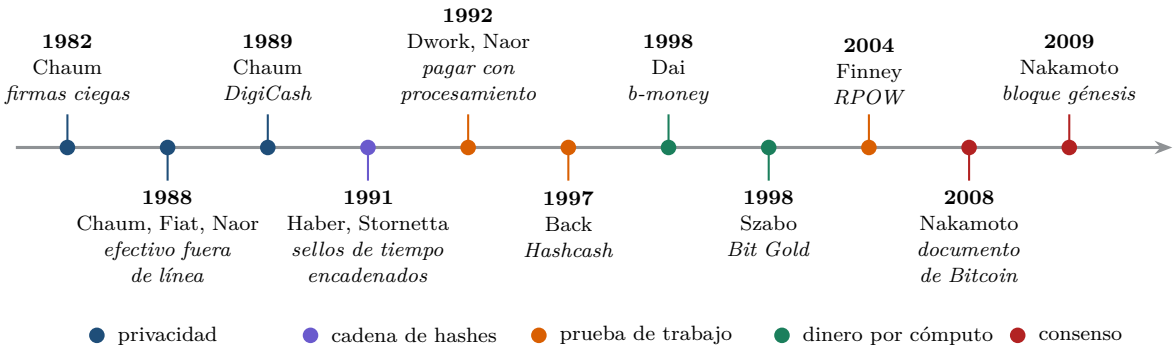


Figura 2. Once hitos entre 1982 y 2009 (escala no proporcional al tiempo). El color indica el aporte principal de cada propuesta.

Datos: `data/timeline.csv`

2 Antecedentes: las piezas que llegaron antes

El problema no se resolvió de un día para otro. Durante un cuarto de siglo, distintos criptógrafos aportaron piezas sueltas: anonimato, sellos de tiempo encadenados, rompecabezas costosos, dinero creado por cómputo. Ninguna, por sí sola, armaba el cuadro completo.

Las fechas de Bitcoin son parte de la historia: el documento apareció el 31 de octubre de 2008; el bloque génesis tiene fecha del 3 de enero de 2009; el software se liberó el 9 de enero; y Hal Finney recibió la primera transacción de la historia el 12 de enero de 2009.

> **Nota 2.1 — Qué hizo falta combinar** — Visto en retrospectiva, cada pieza resuelve una parte: las *firmas* identifican al emisor, las *cadenas de hashes* vuelven costoso alterar el pasado, la *prueba de trabajo* hace costoso producir historia, y un *registro replicado* elimina el punto central. Lo que faltaba era una regla para que, ante historias en competencia, todos los participantes honestos elijan la misma sin consultar a nadie: la regla de la cadena con más trabajo acumulado (sección 7).

3 La idea de 2008: una CPU, un voto

El aporte de Nakamoto no fue inventar la criptografía de hash ni la prueba de trabajo —ambas ya existían— sino darles un uso nuevo. Si cada participante tuviera un voto, bastaría crear miles de identidades falsas para controlar la votación (un *ataque Sybil*). En cambio, el documento propone que se vote con capacidad de cómputo: «una CPU, un voto».

> **Definición 3.1 — Ataque Sybil** — Un ataque Sybil consiste en presentarse con muchas identidades falsas para obtener más influencia de la que corresponde a un solo participante. Un sistema es *resistente a Sybil* si fabricar identidades adicionales no aumenta el poder de quien las fabrica.

Falsificar identidades es gratis. Falsificar poder de cómputo no lo es: requiere comprar hardware real y pagar electricidad real. Esa es la base de todo lo que sigue.

> **Resultado 3.1 — El voto es proporcional al cómputo** — Si cada intento de hash es un ensayo independiente con la misma probabilidad de éxito, un minero que aporta una fracción f del total de hashes por segundo encuentra el siguiente bloque con probabilidad exactamente

Año	Propuesta	Aporte	Lo que le faltaba
1982–83	Chaum: firmas ciegas [2]	Un banco que firma monedas sin ver su número de serie: dinero digital con privacidad.	El banco, en línea, revisaba cada moneda contra su lista de monedas usadas: el problema se delegaba, no se eliminaba.
1988–90	Chaum, Fiat, Naor: efectivo fuera de línea; DigiCash [3]	La moneda funciona sin consultar al banco; si alguien la gasta dos veces, las dos transacciones juntas revelan su identidad.	El doble gasto no se impide: se detecta después y se castiga. DigiCash (fundada en 1989–90) quebró en 1998.
1991	Haber y Stornetta: sellos de tiempo encadenados [4]	Cada sello incluye el hash del anterior: alterar uno obliga a alterar todos los posteriores. Es la estructura de la cadena de bloques.	Aún dependían de un servicio de sellado o de testigos. El documento de Bitcoin cita tres trabajos suyos.
1992	Dwork y Naor: «pagar con procesamiento» [5]	Primera propuesta formal de exigir un cálculo costoso, pero fácil de verificar, para enviar un correo y combatir el spam. Es la idea de la prueba de trabajo.	No era dinero; sirve solo como filtro contra el abuso.
1997	Back: Hashcash [6]	Hallar un texto cuyo hash (SHA-1 en la versión original) empiece con cierta cantidad de bits en cero. Es el rompecabezas que Bitcoin adoptó casi tal cual, ahora con doble SHA-256.	No prevenía el doble gasto. El término «prueba de trabajo» lo acuñaron Jakobsson y Juels en 1999 [7].
1998	Dai: b-money [8]	Participantes que crean dinero resolviendo problemas computacionales y mantienen colectivamente un registro de cuentas.	Un mecanismo concreto para que una red de desconocidos acuerde ese registro. Nakamoto lo cita como primera referencia.
1998/2005	Szabo: Bit Gold [9]	Encadenar soluciones de prueba de trabajo, cada una dependiente de la anterior, para crear escasez digital verificable.	Un consenso para decidir qué historia es la válida cuando hay versiones en competencia.
2004	Finney: RPOW [10]	Pruebas de trabajo reutilizables: fichas de Hashcash que un servidor puede transferir entre usuarios.	El servidor evita el doble gasto, así que vuelve a ser un punto central (aunque auditable).
2008–09	Nakamoto: Bitcoin [1]	Usar la prueba de trabajo no como peaje, sino como mecanismo de *consenso*.	—

Tabla 1. Antecedentes de Bitcoin: qué aportó cada propuesta y qué le faltaba para resolver el doble gasto sin autoridad central.

f, sin importar cuántas identidades use para hacerlo. (Demostración en el Apéndice C, resultado C.1.)

Ojo 3.1 — «Una CPU, un voto» es el diseño de 2008 — La frase describe el diseño original. Desde 2013 la minería se hace con circuitos especializados (ASIC), mucho más eficientes que una CPU, así que hoy es más exacto decir «un hash, un voto». El principio sigue intacto: el voto cuesta recursos físicos.

4 La herramienta: qué es un hash

Una función hash criptográfica como SHA-256 convierte cualquier entrada en 256 bits, que se escriben como 64 caracteres hexadecimales. Importan tres propiedades:

- **Determinista:** la misma entrada da siempre la misma salida.
- **Efecto avalancha:** cambiar un solo carácter cambia, en promedio, la mitad de los 256 bits de salida, sin ningún patrón aprovechable.
- **Resistencia a preimagen:** dado un hash, encontrar una entrada que lo produzca es computacionalmente inviable; no se conoce un método mejor que probar candidatos, y hay $2^{256} \approx 1.16 \times 10^{77}$ salidas posibles. No es «imposible» en sentido matemático, sino que está fuera del alcance de cualquier cómputo físico concebible.

Definición 4.1 — SHA-256 y SHA-256d — SHA-256 es la función hash que a cualquier cadena de bytes m le asigna un valor de 256 bits. Bitcoin utiliza su versión *doble*,

$$\text{SHA-256d}(m) \;=\; \text{SHA-256}\big(\text{SHA-256}(m)\big),$$

y no la aplica a un texto, sino a un encabezado binario de exactamente 80 bytes (sección 5).

4.1 El efecto avalancha, en números

Para medirlo se comparan dos entradas casi idénticas. Por ejemplo, las cadenas «`prueba de trabajo`» y «`prueba de trabaio`» (un solo carácter distinto) producen

```
SHA-256(A)   47743df0939a0429477463cc0b5cd06b33e343f4761b949aeb3fa6bc6f18483e
SHA-256(B)   cc7cef41670a0df1c90d0fe8865e503c07465f2ab3440d67f40ef2de683309fa
```

que difieren en 115 de los 256 bits. Con doble SHA-256 los resultados son tan distintos como antes (135 bits de diferencia). Para ver que no es una coincidencia basta preguntarse cuántos bits difieren *en promedio*: si cada bit de salida cambiara de forma independiente con probabilidad $\frac{1}{2}$, el número de bits distintos seguiría una distribución binomial con $n = 256$ y $p = \frac{1}{2}$:

$$X \sim \text{Bin}\big(256, \tfrac{1}{2}\big), \qquad \text{E}[X] = 128, \qquad \sqrt{\text{Var}[X]} = \sqrt{256 \cdot \tfrac{1}{4}} = 8.$$

Verificación numérica 4.1 — 20 000 pares de entradas — Se generaron 20 000 mensajes aleatorios de 32 bytes y, para cada uno, se alteró un único bit al azar. Se calculó SHA-256

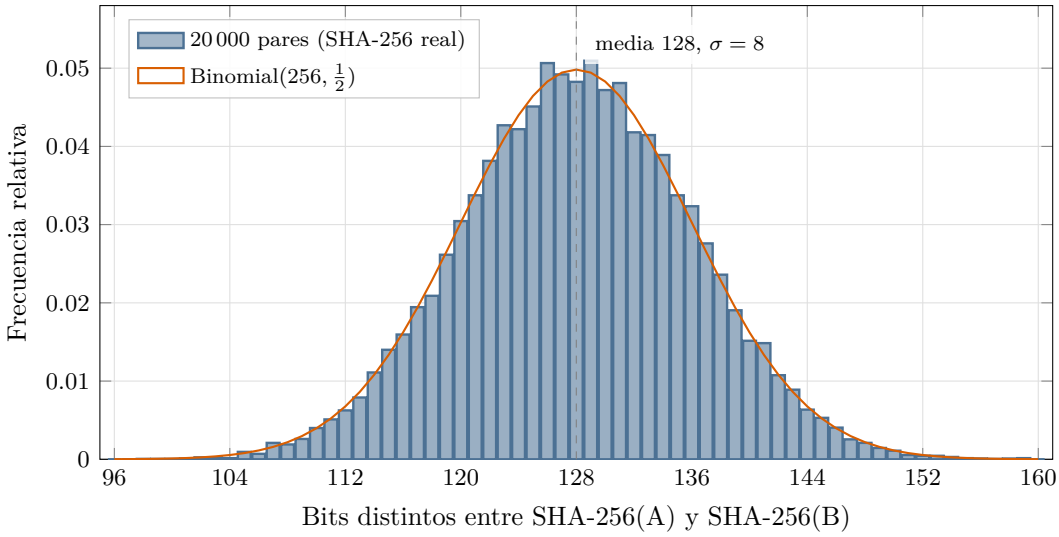


Figura 3. Bits que cambian en la salida de SHA-256 al alterar un solo bit de la entrada. Las barras son el experimento con SHA-256 real; la curva es la binomial teórica.

Datos: `data/avalanche_hist.csv`

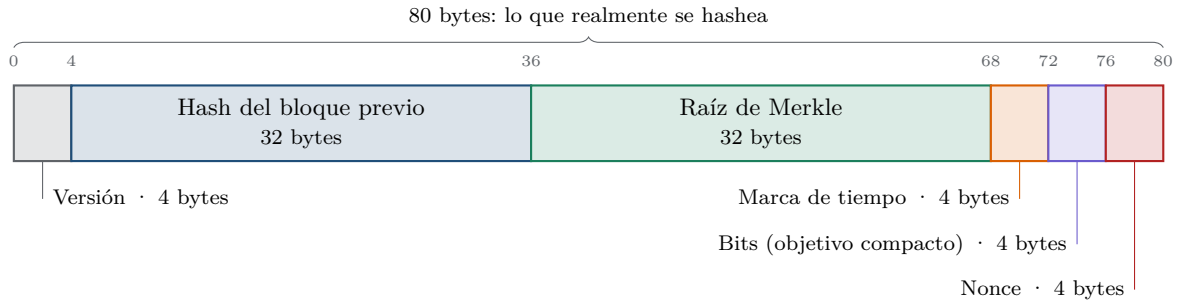


Figura 4. Encabezado de bloque de Bitcoin: seis campos, 80 bytes. La barra es proporcional al tamaño de cada campo; los números son desplazamientos en bytes.

Datos: `data/genesis_header.json`

> de ambas versiones y se contaron los bits distintos de la salida. Resultado: media 128.08, desviación estándar 7.94, mínimo 98 y máximo 159, prácticamente iguales a los valores teóricos 128 y 8 (figura 3).

La consecuencia práctica es que no hay forma de «acercarse» a un hash deseado: cada candidato nuevo es un sorteo independiente. Esa es la propiedad que convierte la búsqueda de un nonce en un juego de azar puro (sección 6).

5 Anatomía de un bloque real

Lo que un minero hashea no son las transacciones completas, sino un encabezado de exactamente 80 bytes con seis campos. Las transacciones entran a través de un único resumen, la *raíz de Merkle*: modificar cualquier transacción cambia esa raíz y, con ella, el hash del bloque.

Campo	Bytes	Valor	En el encabezado
Versión	4	1	`01000000`
Hash del bloque previo	32	64 ceros (no hay bloque previo)	64 ceros
Raíz de Merkle	32	`4a5e1e4baab89f3a32518a88c31bc87f618f76673e2cc77ab2127b7afdeda33b`	`3ba3edfd7a7b12b27ac72c3e67768f617fc81bc3888a51323a9fb8aa4b1e5e4a`
Marca de tiempo	4	1 231 006 505 (3 ene. 2009, 18:15:05 UTC)	`29ab5f49`
Bits	4	`0x1d00ffff`	`ffff001d`
Nonce	4	2 083 236 893	`1dac2b7c`

Tabla 2. Encabezado del bloque génesis de Bitcoin (bloque 0).

5.1 El bloque génesis, campo por campo

Los enteros del encabezado se escriben en orden *little-endian* (el byte menos significativo primero) y los hashes con los bytes invertidos respecto a como se muestran; por eso el hash de un bloque se lee «al revés». La tabla 2 muestra el bloque 0 con sus valores reales.

Verificación numérica 5.1 — el hash del bloque génesis — Se concatenaron los 80 bytes de la tabla 2 y se aplicó SHA-256 dos veces. Invirtiendo los bytes del resultado, tal como se muestra en los exploradores, se obtiene

`000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f`

que es el hash oficial del bloque 0. Leído como número de 256 bits tiene 43 bits iniciales en cero.

5.2 El objetivo y el campo «bits»

La regla real no es «empezar con n ceros», sino que el hash, leído como número de 256 bits, sea menor o igual que un *objetivo* (*target*) que fija el campo *bits*. Ese campo es una notación compacta: el primer byte es un exponente e y los otros tres un coeficiente c, de modo que

$$\text{objetivo} \;=\; c \cdot 256^{\,e-3}.$$

Para el génesis, `0x1d00ffff` significa $e = \texttt{0x1d} = 29$ y $c = \texttt{0x00ffff}$, es decir, objetivo $= \texttt{0xffff} \cdot 2^{208}$, o en hexadecimal `00000000ffff` seguido de 52 ceros. Contar ceros iniciales es una aproximación muy buena, porque cada cero hexadecimal adicional divide el objetivo entre 16.

Resultado 5.1 — Trabajo asociado a un objetivo — Como el hash se comporta como un número uniforme en $[0, 2^{256})$, la probabilidad de que un intento cumpla el objetivo T es $p = (T+1)/2^{256}$ y el número esperado de intentos es

$$\mathrm{E}[\text{intentos}] \;=\; \frac{2^{256}}{T+1}.$$

Con el objetivo del génesis resulta $4\,295\,032\,833$, es decir, $2^{32.00002}$: la *dificultad 1* equivale a unos $2^{32} \approx 4.3 \times 10^{9}$ hashes.

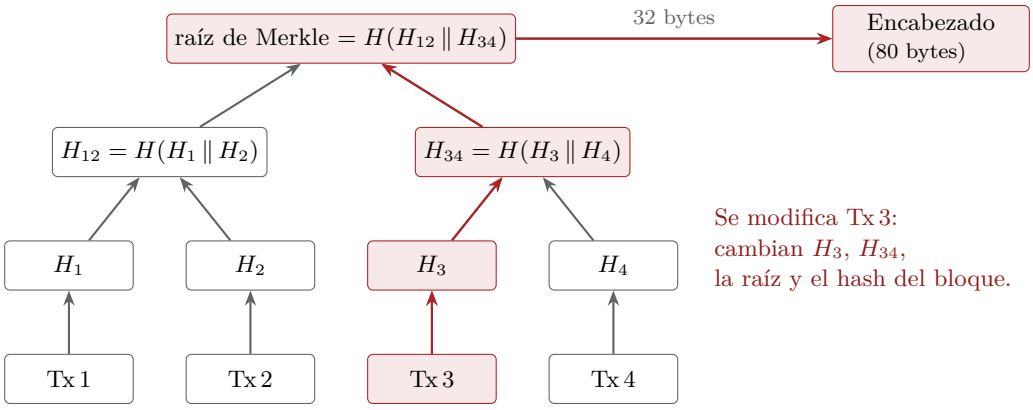


Figura 5. Árbol de Merkle de cuatro transacciones. En rojo, el camino que cambia si se altera la transacción 3: basta una raíz de 32 bytes para comprometer todas las transacciones del bloque.

5.3 La raíz de Merkle

Las transacciones del bloque se resumen en un único valor de 32 bytes mediante un árbol de hashes: cada transacción se hashea, los hashes se emparejan y se hashean de nuevo, y así sucesivamente hasta quedar uno solo, la raíz. Si algún nivel tiene un número impar de elementos, el último se duplica. Con cuatro transacciones:

$$\text{raíz} \;=\; H\big(H(H_1 \,\|\, H_2) \,\|\, H(H_3 \,\|\, H_4)\big), \qquad H_i = \text{SHA-256d}(\text{Tx}_i).$$

Ojo 5.1 — El nonce solo tiene 32 bits — El nonce ocupa 4 bytes: unos 4 300 millones de valores. Un equipo de minería moderno, que calcula cientos de billones de hashes por segundo (cientos de TH/s), los agota en unas decenas de microsegundos ($4.3 \times 10^9 / 2 \times 10^{14} \approx 21\ \mu$s a 200 TH/s). Por eso, además, se varía un *extranonce* dentro de la transacción de recompensa —lo que cambia la raíz de Merkle—, y se ajustan la marca de tiempo y algunos bits de la versión.

6 El rompecabezas: minar un bloque

Minar consiste en variar el nonce hasta que el hash del encabezado tenga suficientes bits iniciales en cero. No hay forma inteligente de buscarlo: cada intento tiene la misma probabilidad de éxito, 2^{-d} si se exigen d bits en cero, independientemente de los intentos anteriores. Por eso el número esperado de intentos es 2^d.

Resultado 6.1 — Intentos hasta el éxito — Sea N el número de intentos hasta el primer éxito, con probabilidad de éxito $p = 2^{-d}$ por intento. Entonces N tiene distribución geométrica,

$$\mathrm{P}(N > n) = (1-p)^n \approx e^{-n/2^d}, \qquad \mathrm{E}[N] = 2^d.$$

Medido en unidades del valor esperado, $T = N/2^d$ es, con excelente aproximación, una variable exponencial de media 1: la mediana es $\ln 2 \approx 0.693$ y la probabilidad de superar la media es $e^{-1} \approx 36.8\,\%$. (Apéndice C, resultado C.2.)

Cantidad (en unidades de 2^{14} intentos)	Medido (600 bloques)	Teoría
Media de intentos	0.946 ($\pm$0.041)	1
Mediana	0.724	$\ln 2 = 0.693$
Desviación estándar	0.883	1
Fracción que tardó más que la media	36.2 %	$e^{-1} = 36.8\,\%$
$\mathrm{P}(T > 2 \mid T > 1)$ (sin memoria)	33.2 %	$e^{-1} = 36.8\,\%$
Mayor valor observado	5.89	—

Tabla 3. Minería real de 600 bloques con $d = 14$ frente a la distribución exponencial. La media medida cae a poco más de un error estándar de 1 ($1/\sqrt{600} = 0.041$).

> **Nota 6.1 — La minería no tiene memoria** — Como cada intento es independiente, llevar mucho tiempo buscando no acerca el éxito: $\mathrm{P}(N > n + m \mid N > n) \approx \mathrm{P}(N > m)$. Quien lleva horas sin hallar un bloque está exactamente igual de lejos que al empezar.

6.1 Cómo calcula el hardware: el *midstate*

El encabezado mide 80 bytes y SHA-256 procesa bloques de 64 bytes. Los primeros 64 bytes no incluyen el nonce, así que su estado intermedio (*midstate*) se calcula una sola vez. Para cada nonce solo se procesan los 16 bytes restantes —que incluyen el nonce— y después el segundo SHA-256:

```
function minar(encabezado76, d):
    mid <- estado_SHA256(encabezado76[0..63])       // se calcula UNA sola vez
    for nonce = 0 .. 2^32 - 1:
        bloque2 <- encabezado76[64..75] || nonce_LE(nonce) || relleno
        h1 <- continuar_SHA256(mid, bloque2)
        h2 <- SHA256(h1)
        if entero_LE(h2) < 2^(256 - d):             // d bits iniciales en cero
            return nonce
```

6.2 Un experimento con minería real

Para comprobar el Resultado 6.1 se minaron 600 bloques con SHA-256 doble real sobre encabezados de 80 bytes, exigiendo $d = 14$ bits iniciales en cero (16 384 intentos esperados por bloque). En total fueron 9 297 759 hashes, calculados en Python a unos 0.69 MH/s.

6.3 La escala: de tu navegador a la red de Bitcoin

La figura 7 pone en la misma gráfica cuánto tardaría en promedio un navegador (alrededor de 1 MH/s) y la red completa de Bitcoin (alrededor de 1 ZH/s[1], a septiembre de 2026) en hallar un hash con d bits iniciales en cero. Con 1 MH/s, $d \approx 20$ bits se alcanzan en aproximadamente un segundo. La red, en cambio, necesita unos diez minutos para $d \approx 79$ bits: $10^{21} \times 600\ \mathrm{s} = 6 \times 10^{23} \approx 2^{79}$ intentos esperados. La red es del orden de mil billones (10^{15}) de veces más rápida que un navegador: lo mismo que haría el navegador en unos 1.9×10^{10} años.

[1] Un zetta-hash por segundo son 10^{21} hashes por segundo: mil trillones.

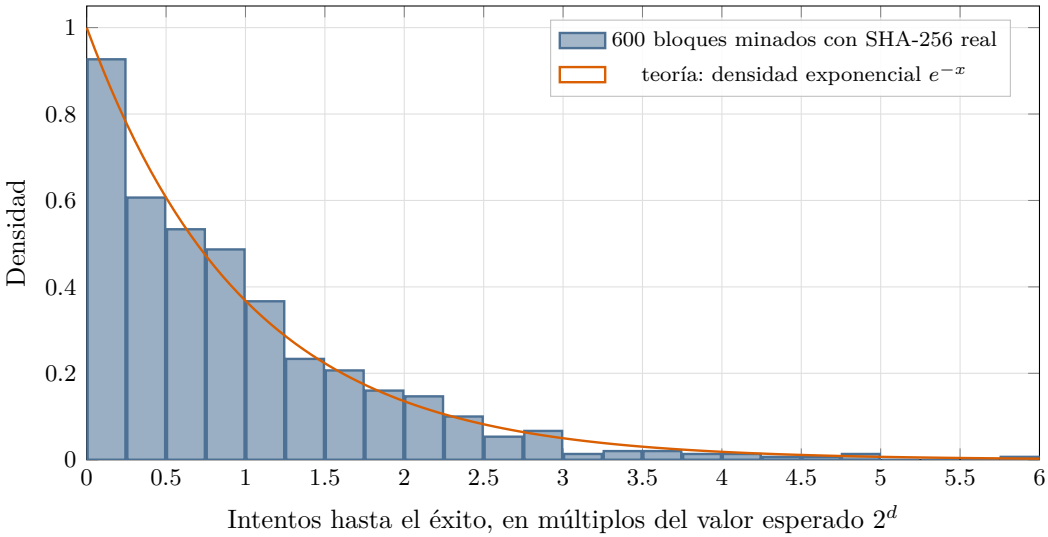


Figura 6. Distribución del esfuerzo necesario para minar un bloque ($d = 14$). La mayoría de los bloques sale con menos esfuerzo que el promedio, y unos pocos exigen varias veces el valor esperado.

Datos: `data/mining_hist.csv`

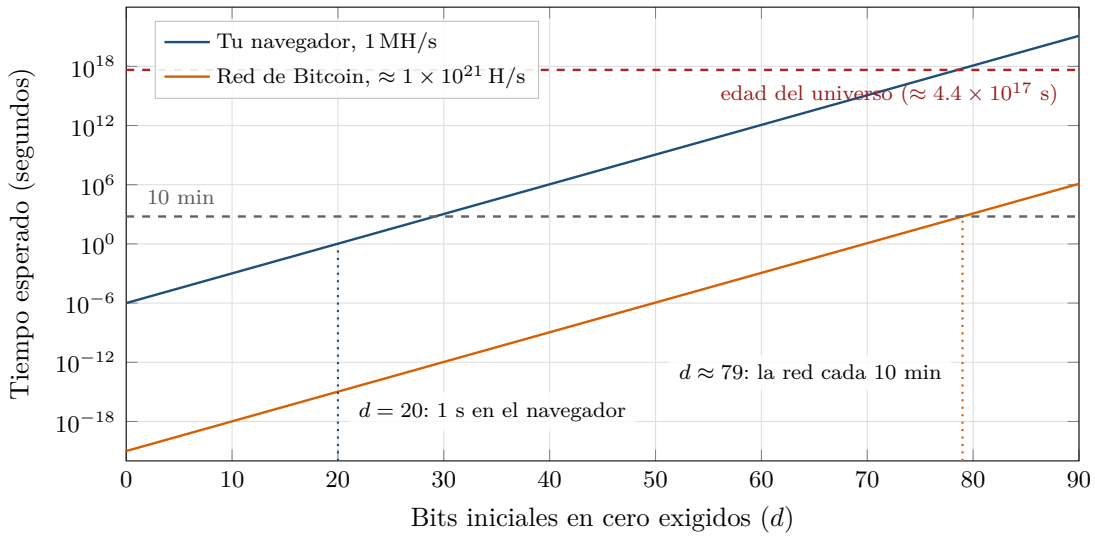


Figura 7. Tiempo esperado para hallar un hash con d bits iniciales en cero. El eje vertical es logarítmico: cada bit adicional duplica el trabajo.

Datos: `data/difficulty_scale.csv`

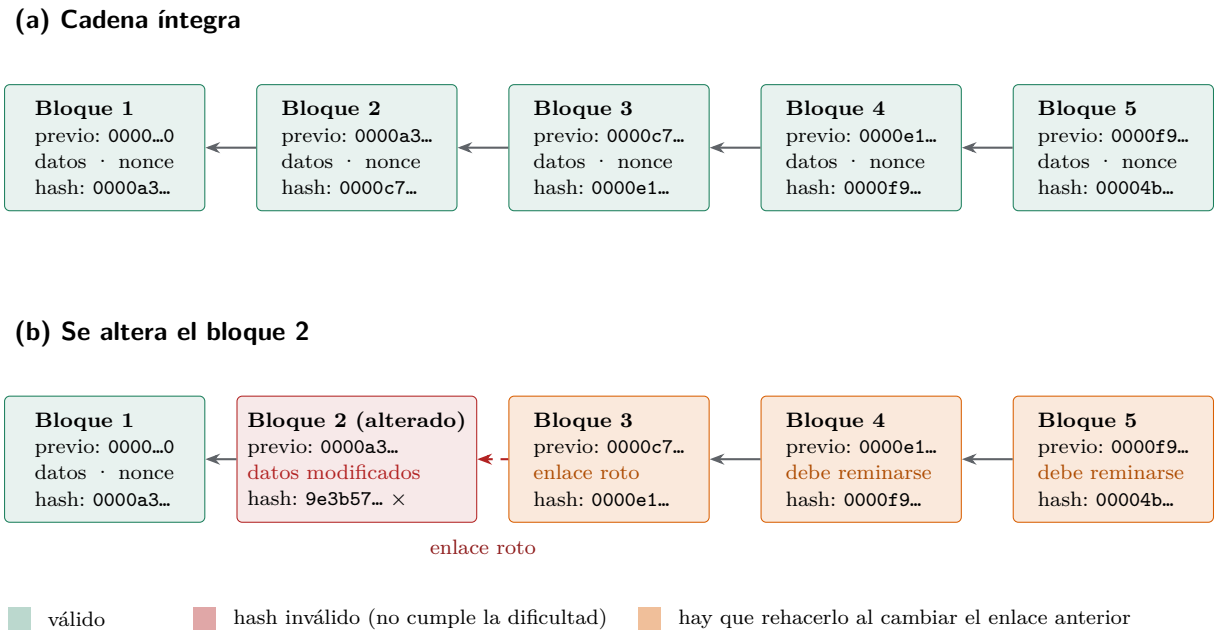


Figura 8. Una cadena de cinco bloques. Alterar el bloque 2 invalida su hash y, al cambiar su valor, obliga a rehacer los bloques 3, 4 y 5, cada uno con su propia prueba de trabajo. Los hashes son ilustrativos.

> **Nota 6.2 — Una máquina frente a la red** — Una sola máquina ASIC actual calcula cientos de billones de hashes por segundo (cientos de TH/s). Aun así, la red completa es mucho mayor: con 200 TH/s por máquina harían falta unos cinco millones de máquinas para igualar 10^{21} H/s.

7 Por qué encadenar: la cadena con más trabajo gana

Cada bloque incluye el hash del anterior. Si alguien altera una transacción antigua, el hash de ese bloque cambia, deja de cumplir la dificultad y, además, rompe el enlace con el bloque siguiente, que apuntaba al hash viejo. Repararlo exige volver a minar ese bloque y todos los posteriores.

7.1 La regla de consenso

Los nodos reconocen como válida la cadena con la mayor cantidad acumulada de trabajo, es decir, la suma, sobre todos sus bloques, del número esperado de hashes que costó producirlos. Casi siempre coincide con la «más larga», pero no es lo mismo: una cadena con muchos bloques fáciles no gana a una con menos bloques más difíciles. De hecho, el documento de 2008 habla de «la cadena más larga»; el software pronto pasó a comparar trabajo acumulado, precisamente para cerrar esa puerta.

> **Definición 7.1 — Trabajo acumulado de una cadena** — Si un bloque tiene objetivo T_i, su *trabajo* es el número esperado de hashes para producirlo, $w_i = 2^{256}/(T_i+1)$ (Resultado 5.1). El trabajo de una cadena de n bloques es
>
> $$W = \sum_{i=1}^{n} w_i,$$
>
> y los nodos adoptan la cadena válida con mayor W.

Ejemplo 7.1 — Más larga no es lo mismo que con más trabajo — Una cadena A tiene 10 bloques de dificultad 1: $W_A = 10 \cdot 2^{32} \approx 4.3 \times 10^{10}$. Una cadena B tiene solo 4 bloques, pero de dificultad 4: $W_B = 4 \cdot 4 \cdot 2^{32} \approx 6.9 \times 10^{10}$. Gana B, aunque sea más corta. (Ejemplo ilustrativo: en la práctica la dificultad solo cambia cada 2 016 bloques, sección 8.)

7.2 Dos formas de decidir quién llegó primero

Con un tercero de confianza — Un banco decide qué transacción llegó primero. Su palabra es la verdad porque es la única que existe.

Con prueba de trabajo — Nadie decide: se reconoce como verdad la historia que costó más trabajo producir, algo que cualquiera puede verificar de forma independiente con un cálculo barato.

8 El reloj: diez minutos por bloque, pase lo que pase

Si el poder de cómputo de la red se duplica, los bloques saldrían el doble de rápido. Para evitarlo, cada 2 016 bloques (unas dos semanas) todos los nodos recalculan el objetivo con una regla simple: si los últimos 2 016 bloques tardaron menos de lo previsto, la dificultad sube en la misma proporción; si tardaron más, baja. El ajuste está acotado a un factor de 4 por periodo.

Resultado 8.1 — Regla de ajuste de dificultad — Sea D la dificultad del periodo que termina y T_{real} el tiempo que tardaron sus 2 016 bloques. La dificultad del siguiente periodo es

$$D_{\text{nueva}} \;=\; D \cdot \frac{2\,016 \times 10 \text{ min}}{T_{\text{real}}},$$

con el factor $\frac{2\,016 \times 10\,\text{min}}{T_{\text{real}}}$ limitado al intervalo $\left[\frac{1}{4},\, 4\right]$. Con el poder de cómputo H constante, el tiempo esperado por bloque es $10 \text{ min} \cdot D/H$ (en unidades relativas).

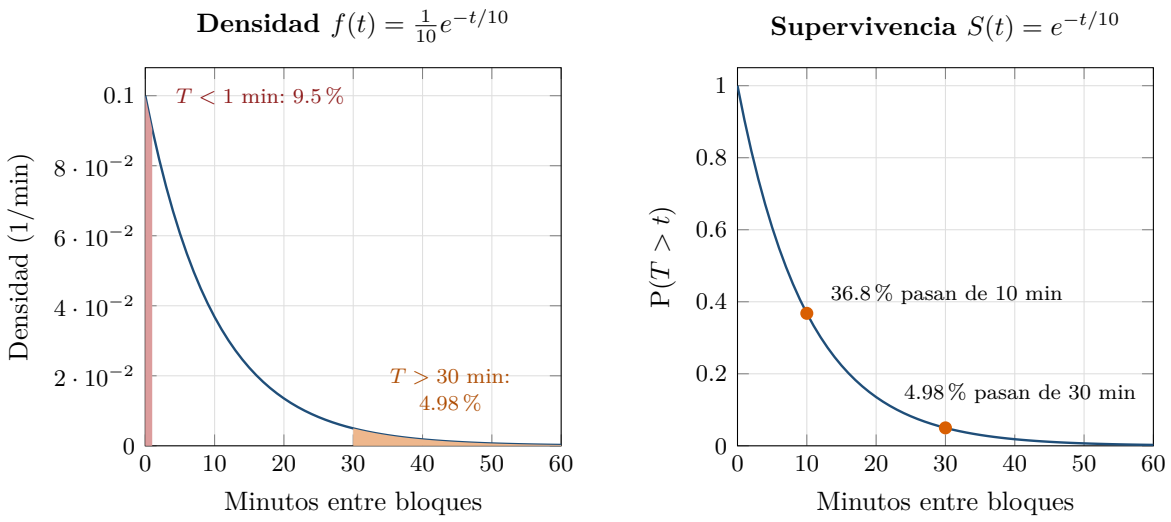
Nota 8.1 — Nota de implementación — En la implementación de referencia, T_{real} se mide entre la primera y la última marca de tiempo del periodo, es decir, sobre 2 015 intervalos y no 2 016. El efecto es de apenas unas décimas de punto porcentual y no altera nada de lo que sigue.

8.1 Los tiempos entre bloques son exponenciales

Aun con el poder de cómputo constante, los bloques no salen cada diez minutos exactos: el tiempo entre bloques es una variable exponencial de media 10 minutos (sección 6). De ahí salen cifras que sorprenden:

$$\begin{aligned} \mathrm{P}(T > 30 \text{ min}) &= e^{-3} \approx 4.98\,\%, \\ \mathrm{P}(T < 1 \text{ min}) &= 1 - e^{-0.1} \approx 9.5\,\%, \\ \mathrm{P}(T > 60 \text{ min}) &= e^{-6} \approx 0.25\,\%. \end{aligned}$$

Es decir, uno de cada veinte bloques tarda más de media hora y alrededor del 10 % sale en menos de un minuto: es la naturaleza exponencial de la búsqueda, no una falla de la red.



(a) Densidad del tiempo entre bloques.

(b) Probabilidad de tardar más de t.

Figura 9. Tiempo entre bloques con poder de cómputo constante: exponencial de media 10 minutos.

Datos: `data/block_time_dist.csv`

8.2 Simulación del ajuste

El simulador de la página original genera cada bloque como una variable exponencial con media $10\ \text{min} \cdot D/H$. Aquí se hace lo mismo periodo por periodo: la suma de 2 016 tiempos exponenciales sigue una distribución gamma; el tiempo medio por bloque de cada periodo se obtiene de ella y se aplica la regla del Resultado 8.1. Se comparan cuatro escenarios, *con* y *sin* ajuste, usando los mismos números aleatorios en ambos casos (figura 10).

Lo que muestran los cuatro escenarios:

- **Crecimiento sostenido (+6 % por periodo).** Sin ajuste, el tiempo por bloque se desplomaría hasta $10/1.06^{25} \approx 2.3$ minutos al cabo de 26 periodos. Con ajuste, solo se retrasa un periodo respecto al crecimiento.
- **Caída súbita del 50 %** (como la de junio de 2021, citada en la página original). El primer periodo posterior a la caída tarda el doble por bloque: unos 20 minutos, lo que equivale a $2\,016 \times 20\ \text{min} \approx 28$ días. Después el ajuste restablece los 10 minutos.
- **Duplicación súbita.** El primer periodo saldría a unos 5 minutos por bloque (la mitad); el ajuste lo corrige.
- **Poder de cómputo constante.** Aun así el tiempo medio fluctúa un poco de un periodo a otro (véase la nota siguiente).

> **Nota 8.2 — ¿Cuánto fluctúa un periodo de 2 016 bloques?** — La suma de 2 016 exponenciales de media 10 minutos tiene media 20 160 min = 14 días y desviación relativa $1/\sqrt{2\,016} \approx 2.2\,\%$, es decir, unas 7.5 horas. Por eso, incluso sin cambios en el hardware, la dificultad se mueve alrededor de $\pm 2\,\%$ de un periodo al siguiente.

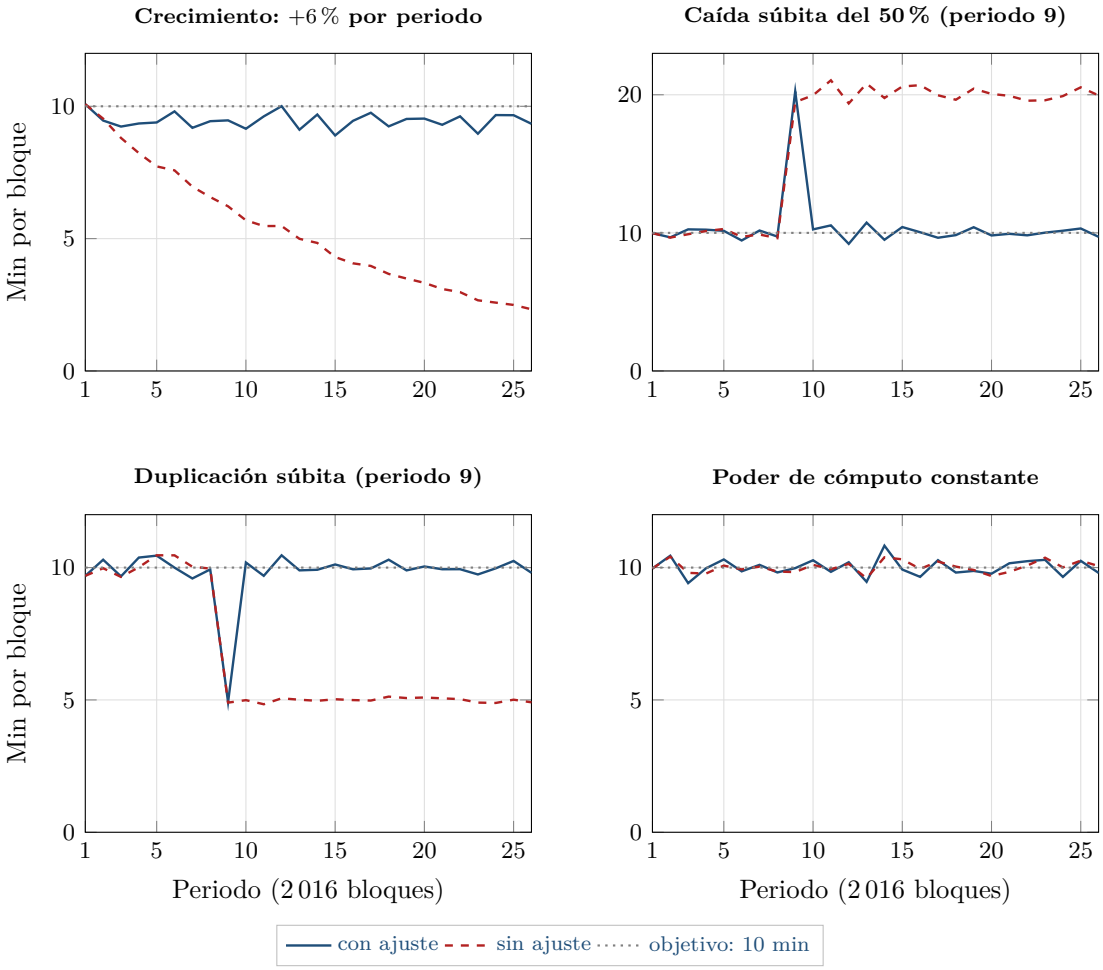


Figura 10. Tiempo medio por bloque en cada periodo de 2 016 bloques, con y sin ajuste de dificultad. Con ajuste, el reloj regresa a 10 minutos en uno o dos periodos; sin él, el sistema queda acelerado o frenado para siempre.

Datos: `data/difficulty_sim.csv`

9 Cómo se intentaría el doble gasto hoy

Imagina que alguien paga a un comerciante y, al mismo tiempo, empieza a minar en secreto una cadena alternativa a partir del bloque anterior a ese pago, una en la que la transacción nunca ocurrió (o en la que el mismo dinero va a otra dirección suya). El comerciante, prudente, espera z confirmaciones —z bloques en la cadena pública a partir del pago, contando el bloque que lo incluye— antes de entregar la mercancía.

Mientras tanto, el atacante sigue minando su cadena privada. Si alguna vez logra que su cadena acumule más trabajo que la pública, la publica de golpe: los nodos la adoptan por ser la de mayor trabajo, y el pago original desaparece de la historia. Ahí está el doble gasto.

9.1 El modelo

Sean q la fracción del poder de cómputo del atacante y $p = 1 - q$ la de la red honesta. Cada bloque nuevo lo encuentra la red honesta con probabilidad p y el atacante con probabilidad q. Nakamoto modeló la carrera en dos etapas:

1. **Mientras la red honesta produce sus z bloques**, el atacante produce K bloques.
2. **Después**, la diferencia entre ambas cadenas es una caminata aleatoria: sube con probabilidad p (la red honesta encuentra un bloque) y baja con probabilidad q (lo encuentra el atacante).

Resultado 9.1 — Ruina del jugador — Si el atacante va k bloques atrás, la probabilidad de que alguna vez alcance a la red honesta es

$$\mathrm{P}(\text{alcanzar desde } k \text{ atrás}) \;=\; \begin{cases} (q/p)^k, & q < p, \\ 1, & q \geq p. \end{cases}$$

(Demostración en el Apéndice C, resultado C.3.)

Resultado 9.2 — Los bloques del atacante siguen una binomial negativa — El número K de bloques que encuentra el atacante antes de que la red honesta acumule z tiene distribución binomial negativa:

$$\mathrm{P}(K = k) \;=\; \binom{z + k - 1}{k} p^z \, q^k, \qquad k = 0, 1, 2, \ldots$$

con $\mathrm{E}[K] = zq/p$ y $\mathrm{Var}[K] = zq/p^2$.

9.2 Tres cálculos de la misma probabilidad

La pregunta es cuánto vale K. Aquí conviene corregir una idea muy difundida: Nakamoto aproximó K con una distribución de Poisson de media $\lambda = zq/p$. Esa aproximación **no es exacta**: el valor correcto sigue la binomial negativa del Resultado 9.2, como señalaron Rosenfeld (2012) [11] y Grunspan y Pérez-Marco (2017) [12]. Hay un segundo matiz: Nakamoto cuenta como éxito que el atacante *empate*, pero en la práctica los nodos se quedan con la primera cadena que vieron, así que el atacante necesita *superar* a la red por un bloque. Resultan tres versiones:

Resultado 9.3 — Probabilidad de éxito del atacante con z confirmaciones — Con $r = q/p < 1$:

$$P_{\text{Poisson}}(z) = 1 - \sum_{k=0}^{z} \frac{\lambda^k e^{-\lambda}}{k!}\left(1 - r^{\,z-k}\right), \qquad \lambda = \frac{zq}{p} \qquad \text{(Nakamoto 2008)} \qquad (1)$$

$$P_{\text{empate}}(z) = 1 - \sum_{k=0}^{z-1} \binom{z+k-1}{k} p^z q^k \left(1 - r^{\,z-k}\right) \qquad \text{(exacta; empatar)} \qquad (2)$$

$$P_{\text{superar}}(z) = 1 - \sum_{k=0}^{z} \binom{z+k-1}{k} p^z q^k \left(1 - r^{\,z-k+1}\right) \qquad \text{(exacta; superar)} \qquad (3)$$

Si $q \geq p$, las tres valen 1. En la ecuación (1) el término $k = z$ se anula (su factor es $1 - r^0 = 0$); se conserva por fidelidad a la fórmula original de Nakamoto.

Nota 9.1 — ¿Por qué Poisson subestima el riesgo? — Ambas distribuciones tienen la misma media, zq/p. Pero la varianza de la binomial negativa, zq/p^2, es mayor que la de Poisson, zq/p, por el factor $1/p > 1$. Una cola derecha más pesada significa que las «rachas de suerte» del atacante (valores grandes de K) son más probables de lo que Poisson supone, y esas rachas son justamente las que dominan la probabilidad de éxito.

Ejemplo 9.1 — $q = 10\,\%$ y $z = 6$ — Con las fórmulas anteriores:

Cálculo	Probabilidad de éxito
Poisson, empate (Nakamoto 2008), ecuación (1)	0.024 3 %
Exacta, empate, ecuación (2)	0.059 1 %
Exacta, superar, ecuación (3)	0.011 0 %

Para ataques minoritarios típicos, la aproximación de Poisson *subestima* el riesgo: aquí da 0.024 %, cuando el valor exacto con el mismo criterio de empate es 0.059 %, más del doble (2.4×). Exigir que el atacante *supere* a la red (y no solo empate) reduce la probabilidad a 0.011 %.

10 Simulación: la carrera contra la red honesta

Para comprobar las fórmulas de la sección anterior se simula la carrera bloque a bloque. Se fija el poder de cómputo del atacante q y las confirmaciones z que espera el comerciante, y se repite la carrera miles de veces.

```
function carrera(q, z):
    h <- 0; a <- 0                          // bloques de la red honesta y del atacante
    while h < z:                            // etapa 1: la red honesta produce z bloques
        if aleatorio() < q: a <- a + 1 else: h <- h + 1
    d <- h - a                              // deficit del atacante
    empate <- (d <= 0);  superar <- (d <= -1)
    while d > -1 and d < 60:                // etapa 2: caminata aleatoria del deficit
        if aleatorio() < q: d <- d - 1 else: d <- d + 1
        if d <= 0:  empate <- true
        if d <= -1: superar <- true
    return (empate, superar)
```

El tope de 60 bloques de ventaja se trata como «fracaso»; para $q = 30\,\%$ la probabilidad que se

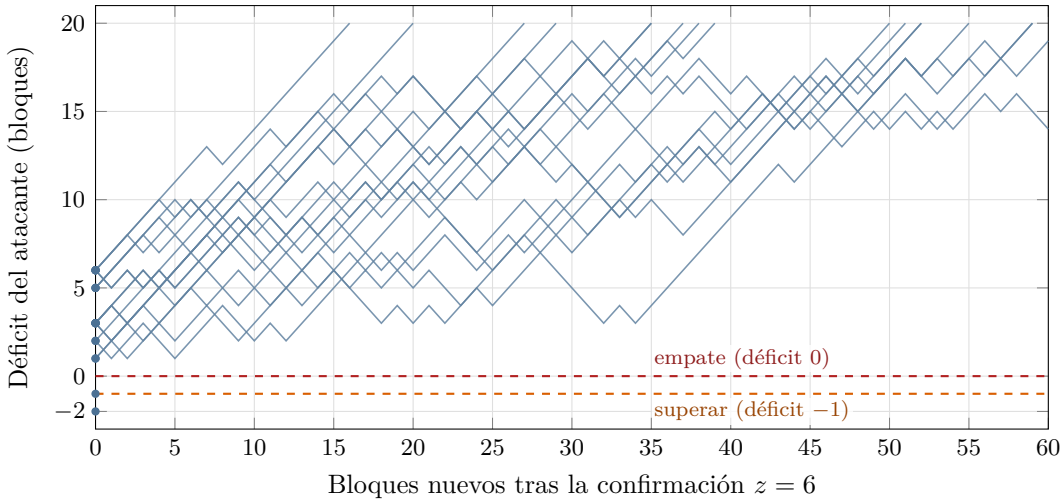


Figura 11. Veinte carreras de ejemplo con $q = 30\,\%$ y $z = 6$ (muestra natural, sin selección). Cada línea parte del déficit con el que el atacante inicia la etapa 2 y termina cuando alcanza el déficit -1 o se aleja demasiado. Dos carreras (puntos aislados en el paso 0) empiezan ya por debajo de cero, es decir, el atacante produjo más de z bloques mientras la red honesta producía z, y logran el doble gasto de inmediato; ninguna otra alcanza la línea punteada. La probabilidad teórica de éxito es 8.9 %, y aquí $2/20 = 10\,\%$.

Datos: `data/race_paths.csv`

		Criterio: empatar		Criterio: superar	
Atacante	**Poisson (2008)**	**Exacta**	**Monte Carlo**	**Exacta**	**Monte Carlo**
$q = 10\,\%$	0.024 3 %	0.059 1 %	0.058 3 % [0.0538; 0.0632]	0.011 0 %	0.011 7 % [0.0098; 0.0140]
$q = 30\,\%$	13.21 %	15.65 %	15.58 % [15.47; 15.70]	8.91 %	8.88 % [8.79; 8.96]

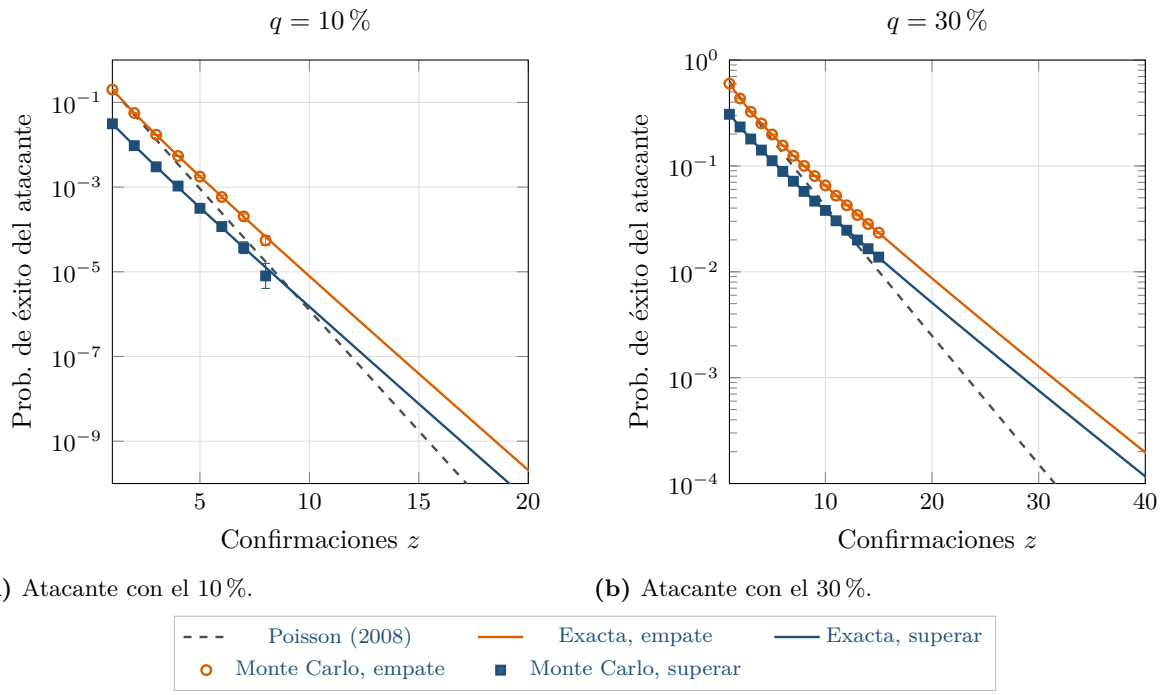
Tabla 4. Probabilidad de éxito del atacante con $z = 6$ confirmaciones. Entre corchetes, el intervalo de confianza al 95 % del Monte Carlo.

descarta es $(q/p)^{60} \approx 8 \times 10^{-23}$.

10.1 El resultado empírico coincide con la fórmula exacta

Se corrieron 1 000 000 de carreras por cada punto con $q = 10\,\%$ ($z = 1, \ldots, 8$) y 400 000 con $q = 30\,\%$ ($z = 1, \ldots, 15$), con intervalos de confianza de Wilson al 95 %. La tabla 4 resume el caso $z = 6$ y la figura 12 muestra todos los puntos.

Verificación numérica 10.1 — ¿Coincide la simulación con la teoría? — En los 23 puntos simulados, el valor exacto del criterio «empatar» cae dentro del intervalo de confianza del 95 % en los 23, y el del criterio «superar» en 22 (uno queda apenas fuera, lo esperable por azar con intervalos al 95 %). La aproximación de Poisson, en cambio, se aleja sistemáticamente del experimento: por ejemplo, con $q = 30\,\%$ y $z = 6$ da 13.2 % cuando la simulación da 15.6 %.



(a) Atacante con el 10 %. **(b)** Atacante con el 30 %.

Figura 12. Probabilidad de que el atacante revierta un pago según las confirmaciones que espera el comerciante (escala logarítmica). Los puntos son simulaciones con intervalo de confianza al 95 %; coinciden con las curvas exactas y no con la aproximación de Poisson.

Datos: `data/double_spend_curves.csv`, `data/double_spend_mc.csv`

10.2 El riesgo, confirmación por confirmación

La probabilidad cae de forma exponencial con cada bloque adicional: para $q = 10\,\%$, el cociente entre dos valores consecutivos de la curva exacta ronda 0.34 ($6.7 \times 10^{-5} / 2.0 \times 10^{-4}$ entre $z = 8$ y $z = 7$); para $q = 30\,\%$ es de 0.82, mucho más lento. Cuanto más cerca del 50 %, más confirmaciones hacen falta (figura 13).

10.3 ¿Cuántas confirmaciones hacen falta?

El documento original incluye una tabla con el mínimo de confirmaciones para que la probabilidad de éxito del atacante baje de 0.1 %. La tabla 5 la recalcula con los tres criterios. La columna de Nakamoto reproduce su tabla de 2008; las otras dos muestran cuánto cambian los números al hacer el cálculo exacto.

Con $q = 30\,\%$, Nakamoto indicaba 24 confirmaciones; el cálculo exacto con empate pide 32 y, exigiendo superar, 29. Con $q = 45\,\%$ la diferencia es mayor: 340 frente a 539 y 530. Con $q = 10\,\%$ ocurre lo contrario para el criterio estricto: basta con 4 confirmaciones, una menos que en la tabla original. No conviene, por tanto, resumir que Poisson «siempre subestima»: depende de qué se exija al atacante.

10.4 El ataque del 51 %

La seguridad no es absoluta: es una apuesta cuantificable que mejora exponencialmente con cada bloque, siempre que el atacante tenga menos de la mitad del poder de cómputo. Con la mitad o más, la caminata aleatoria ya no tiene deriva a su favor (Resultado 9.1: la probabilidad de

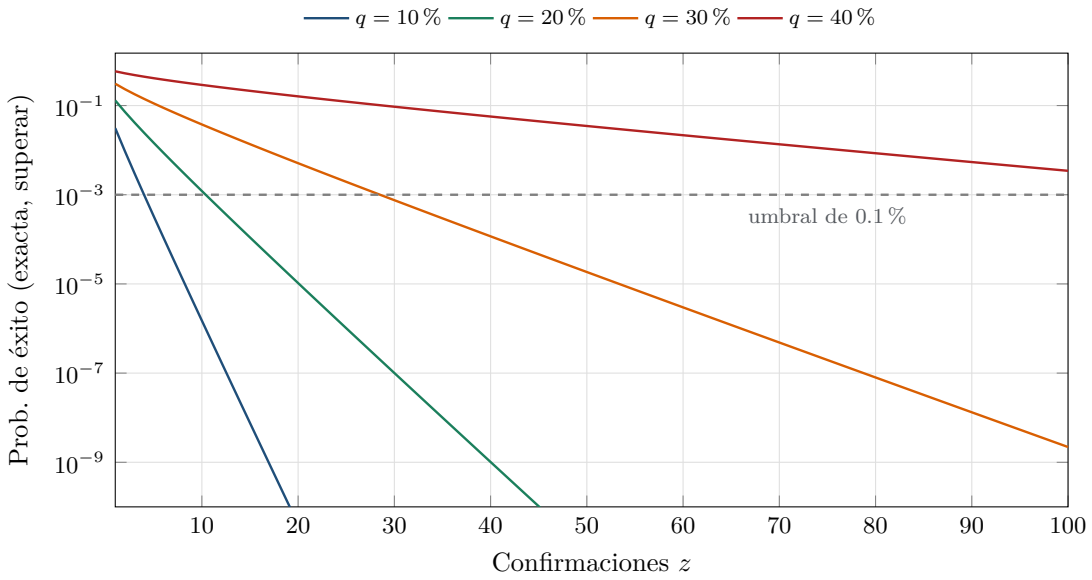


Figura 13. Cómo cae la probabilidad de éxito del atacante al esperar más bloques, para distintos poderes de cómputo del atacante. La línea punteada marca el umbral de 0.1 % usado en la tabla 5.

Datos: `data/double_spend_curves.csv`

Atacante (q)	Poisson, empate (2008)	Exacta, empate	Exacta, superar
0.10	5	6	4
0.15	8	9	7
0.20	11	13	11
0.25	15	20	17
0.30	24	32	29
0.35	41	58	55
0.40	89	133	128
0.45	340	539	530

Tabla 5. Confirmaciones mínimas para que el éxito del atacante sea inferior a 0.1 %.

Datos: `data/confirmations_table.csv`

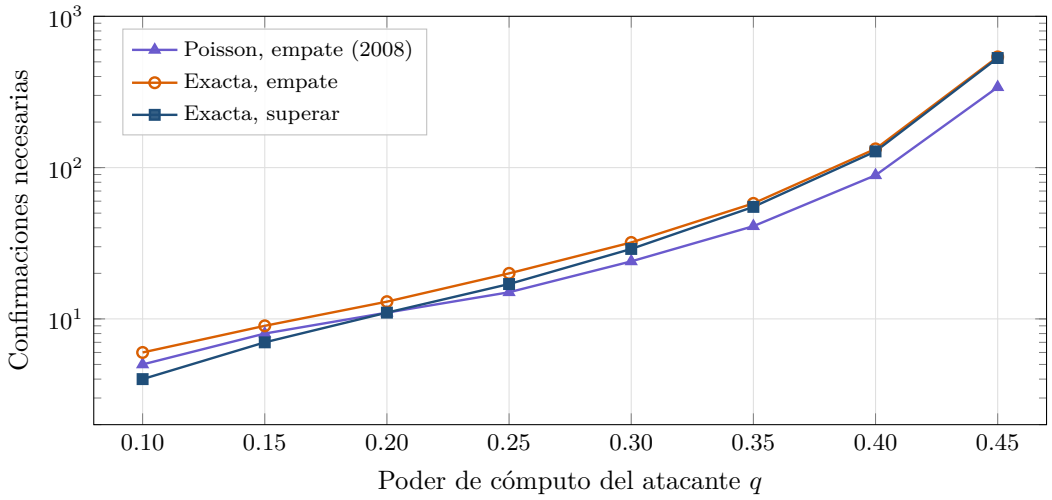


Figura 14. Confirmaciones necesarias para bajar de 0.1 %, según el poder de cómputo del atacante y el criterio de éxito. La escala vertical es logarítmica.

Datos: `data/confirmations_table.csv`

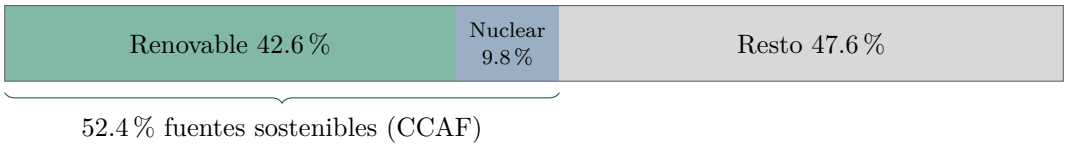


Figura 15. Mezcla energética de la minería de Bitcoin según la encuesta del CCAF (abril de 2025).

Datos: `data/energy_mix.csv`

alcanzar a la red es 1 si $q \geq p$) y el éxito a la larga es seguro: ese es el famoso «ataque del 51 %».

11 Lo que la prueba de trabajo aporta

- **Resistencia a Sybil:** nadie puede fingir tener más votos que poder de cómputo real, porque cada voto cuesta electricidad y hardware (Resultado 3.1).
- **Un orden común sin coordinador:** la red se pone de acuerdo sobre qué transacción fue primero reconociendo la cadena con más trabajo acumulado; la segunda moneda gastada simplemente es inválida (sección 7).
- **Un reloj autorregulado:** el ajuste de dificultad mantiene el ritmo de unos diez minutos por bloque sin importar cuánto hardware entre o salga (sección 8).
- **Un costo creciente y verificable para el fraude:** para un atacante minoritario, la probabilidad de revertir un pago cae exponencialmente con las confirmaciones, y verificar el trabajo de otros cuesta un solo hash (sección 9).

Y lo que **no** garantiza: la seguridad es probabilística, no absoluta; supone que la mayoría del poder de cómputo actúa honestamente; y el modelo simple de este texto ignora la latencia de la red y estrategias más sofisticadas, como la minería egoísta (Eyal y Sirer, 2014) [13], que pueden dar ventaja a mineros con bastante menos del 50 %.

12 El costo de esta solución

La misma prueba de trabajo que da seguridad tiene un precio medible. Las cifras de referencia más citadas provienen del Centro de Finanzas Alternativas de Cambridge (CCAF) [14]:

≈ 138 TW h	consumo eléctrico anual estimado (informe CCAF, abril de 2025), cerca del 0.5 % de la electricidad mundial.
52.4 %	de esa energía proviene de fuentes sostenibles según la encuesta del CCAF: 42.6 % renovable y 9.8 % nuclear.
3.125 BTC	recompensa por bloque desde el cuarto halving (abril de 2024); el siguiente se espera hacia 2028.

Para dimensionar: 138 TW h al año está entre el consumo eléctrico de Bélgica y el de los Países Bajos. Las estimaciones tienen incertidumbre importante; la encuesta del CCAF cubrió empresas que suman cerca de la mitad del poder de cómputo de la red.

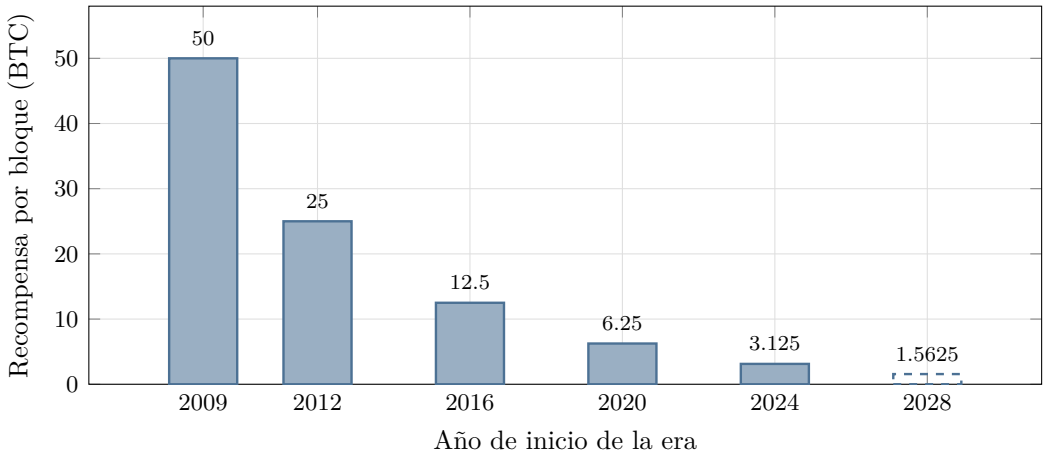


Figura 16. Recompensa por bloque: se divide entre dos cada 210 000 bloques (alrededor de cuatro años). La barra punteada es la proyección para el próximo halving (hacia 2028).

Datos: `data/halving_schedule.csv`

12.1 ¿Cuánto cuesta atacar? Una estimación de orden de magnitud

Si el atacante quiere controlar una fracción q del poder de cómputo total, debe aportar $q/(1-q)$ veces lo que ya tiene la red honesta. Con la potencia actual de la red se puede estimar solo la factura eléctrica del ataque; el hardware, que sería el costo dominante, no está incluido.

Resultado 12.1 — Factura eléctrica del ataque — Con una red honesta de $H_{\text{red}} \approx 1$ ZH/s y $P_{\text{red}} \approx 15.7$ GW (138 TWh/8 760 h, es decir, unos 16 J por TH), un atacante con fracción q necesita

$$H_a = \frac{q}{1-q} H_{\text{red}}, \qquad P_a = \frac{q}{1-q} P_{\text{red}}, \qquad C = P_a \cdot t \cdot \pi,$$

donde t es la duración en horas y π el precio de la electricidad (USD por kWh).

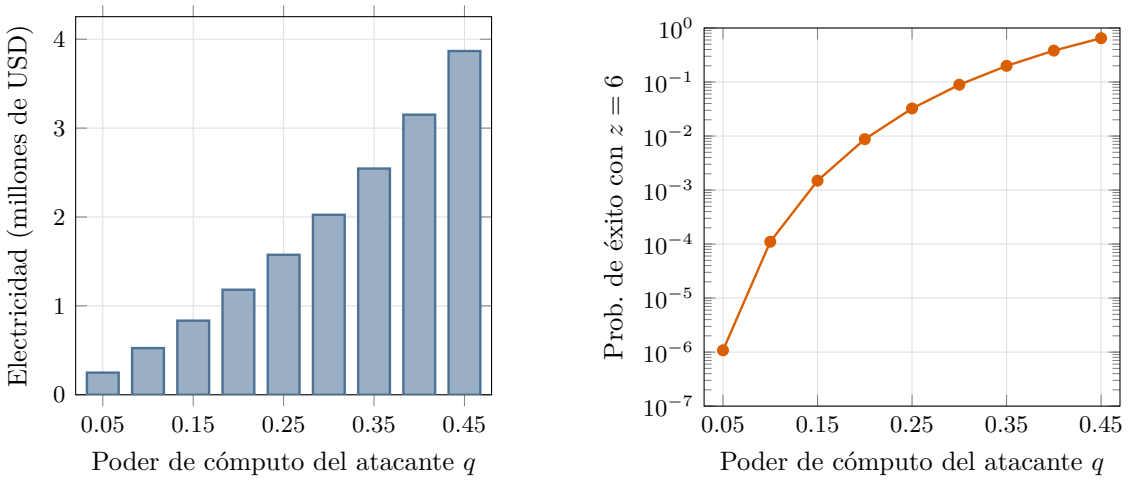
Ejemplo 12.1 — Un atacante con el 30 % durante 6 horas — Con $q = 0.30$, $t = 6$ h y $\pi = 0.05$ USD h/kW: $H_a = 0.43$ ZH/s (428.6 EH/s), $P_a \approx 6.75$ GW y una factura eléctrica de unos 2.03 MUSD (millones de dólares). Con $z = 6$ confirmaciones y criterio de superar, su probabilidad de éxito es 8.9 %.

12.2 Dos posturas sobre si ese gasto vale la pena

Sobre si ese gasto vale la pena hay posturas encontradas, y conviene presentarlas sin inclinar la balanza.

Quienes defienden el diseño argumentan que el gasto energético no es un desperdicio sino el fundamento de la seguridad: un costo real y auditable es lo que hace que reescribir la historia sea económicamente inviable. Señalan además que más de la mitad de la energía proviene de fuentes sostenibles según la encuesta del CCAF, y que la minería puede aprovechar excedentes o ajustarse a la demanda de la red eléctrica, como ocurrió en las tormentas invernales de Texas.

Quienes lo cuestionan señalan el consumo total —comparable al de países europeos medianos—, las emisiones de la parte que sigue usando gas y carbón, la concentración de la minería en pocas empresas y regiones, y proponen alternativas como la *prueba de participación*



(a) Factura eléctrica de 6 horas de ataque.

(b) Probabilidad de éxito con 6 confirmaciones.

Figura 17. Orden de magnitud de un ataque: costo eléctrico de seis horas (supuestos del Resultado 12.1, 0.05 USD h/kW) y probabilidad de éxito con $z = 6$ (criterio de superar).

Datos: `data/attack_cost.csv`

(*proof of stake*), adoptada por Ethereum en 2022, que logra consenso sin gasto energético intensivo, a cambio de otros supuestos de seguridad.

Ambas posturas coinciden en un punto: la prueba de trabajo no elimina la posibilidad de un ataque, la vuelve costosa. Qué tan costosa debe ser, y si ese costo vale la pena frente a otros diseños, sigue siendo un debate abierto en la comunidad técnica y regulatoria.

13 Conclusiones

1. El doble gasto es un problema de *orden*: si todos acuerdan qué transacción fue primera, la segunda es inválida. Con un banco el orden es trivial; sin banco hace falta un mecanismo de acuerdo entre desconocidos.

2. La prueba de trabajo convierte el voto en algo que cuesta recursos físicos, lo que neutraliza los ataques Sybil. El hash criptográfico garantiza que no hay atajos: encontrar un nonce es un proceso de ensayos independientes, con tiempos exponenciales y sin memoria.

3. Al encadenar los bloques mediante hashes, reescribir el pasado exige rehacer todo el trabajo posterior. La regla de la cadena con más trabajo acumulado, junto con el ajuste de dificultad cada 2 016 bloques, da un reloj estable y una historia única.

4. La seguridad contra el doble gasto es probabilística y mejora exponencialmente con las confirmaciones, mientras el atacante tenga menos de la mitad del poder de cómputo. La fórmula correcta es la binomial negativa; la aproximación de Poisson de 2008 subestima el riesgo en el criterio de empate, y el número de confirmaciones recomendable depende del criterio de éxito que se suponga.

5. Esa seguridad tiene un costo energético medible y un debate abierto sobre si merece la pena frente a otros diseños de consenso.

A Guía para convertir este documento en una página web

Este apéndice es el contrato entre el documento y quien (persona o IA) lo convierta a HTML. La página resultante debe reproducir el espíritu de la original: un recorrido narrativo de 12 secciones, *origen* → *problema* → *solución* → *simulación*, con herramientas interactivas calculadas en el navegador.

A.1 Marcado semántico

- `\webhint{...}` no imprime nada en el PDF. Es una lista `clave=valor` separada por «;» cuyo primer elemento es el tipo de bloque (`section`, `figure`, `table`). Debe leerse del código fuente `.tex`.
- Para `section`: `id` (ancla HTML), `slug` (URL), `kicker` (etiqueta en mayúsculas pequeñas sobre el título, p. ej. «06 — el rompecabezas»), `role` (`narrative`, `widget`, `math`, `summary`, `meta`). Las secciones con `render=false` no se publican.
- Para `figure`: `id`, `chart` (tipo de gráfica), `data` (CSV de origen, relativo a la raíz del proyecto), `x` y `y` (columnas), `yscale`, `filter` (qué filas usar), `interactive` (comportamiento deseado) y `alt` (texto alternativo, obligatorio en la página).
- Las cajas `definicion`, `resultado`, `ejemplo`, `verificacion`, `nota` y `cuidado` se convierten en `<aside class="callout callout--TIPO">` con su título. Las que tengan `\label` deben conservar un ancla.

A.2 Correspondencia LaTeX → HTML

LaTeX	HTML / componente
`\section`	`<section id=…>` con `<h2>` y el kicker; menú lateral fijo con las 12 secciones
`\subsection`	`<h3>`
`$…$, equation, align`	KaTeX (las macros propias `\Prob`, `\Esp`, `\Var`, `\SHA`, `\SHAd` deben definirse como macros de KaTeX)
`tabular, tabularx`	`<table>` con `<thead>`, `<caption>` y desplazamiento horizontal en pantallas estrechas
`figure + tikzpicture`	SVG en línea (diagramas) con `<figcaption>` y el `alt` del `\webhint`
`figure + axis (pgfplots)`	Gráfica interactiva (Chart.js, D3 o Plotly) leyendo el CSV indicado en `data`
`lstlisting`	`<pre><code>` con resaltado de sintaxis
`\cite, \ref`	Enlaces a `#ref-clave` y a las anclas de figuras, tablas y secciones
`\SI, \num`	Texto con espacio fino como separador de miles y punto decimal

A.3 Gráficas: qué dibujar y de dónde sale cada número

Todas las gráficas con datos leen un CSV en `data/`. La página web debe cargar esos mismos archivos y *no* copiar cifras a mano.

Figura	Tipo	Datos	Interacción sugerida
1	diagrama	`---`	Selector de modelo (nadie / banco / red) que anima el envío a dos tiendas
2	línea de tiempo	`timeline.csv`	Tarjeta emergente con la aportación; color por categoría
3	barras + línea	`avalanche_hist.csv`	Dos cuadros de texto y cuadrícula de 256 bits con los distintos resaltados
4, 5	diagrama	`genesis_header.json`	Editar campos y recalcular SHA-256d; resaltar el camino de Merkle
6	histograma + curva	`mining_hist.csv`	Botón «minar serie» con dificultad y número de bloques seleccionables
7	líneas, eje log	`difficulty_scale.csv`	Marcadores en $d = 20$ y $d \approx 79$; etiqueta con el tiempo esperado
8	diagrama	`---`	Cadena editable de 5 bloques con 14 bits; «reminar inválidos»
9	área + línea	`block_time_dist.csv`	Colas $T < 1$ min y $T > 30$ min sombreadas
10	líneas 2×2	`difficulty_sim.csv`	Selector de escenario; interruptor con/sin ajuste; «simular 26 periodos»
11	líneas múltiples	`race_paths.csv`	«Simular carreras» dibuja trayectorias nuevas
12	curvas + puntos con IC	`double_spend_curves.csv`, `double_spend_mc.csv`	Sliders de q y z; criterio (empatar / superar); 20 000 / 200 000 / 1 000 000 carreras
13	líneas, eje log	`double_spend_curves.csv`	Resaltar la curva al pasar el cursor
14	líneas con marcadores	`confirmations_table.csv`	Tabla enlazada a la gráfica
15, 16	barras	`energy_mix.csv`, `halving_schedule.csv`	Tooltip con valores
17	barras + línea log	`attack_cost.csv`	Calculadora: q, horas y precio por kWh

A.4 Herramientas interactivas que la página debe incluir

Son las de la página original. Se calculan en el navegador, sin servidor; el SHA-256 puede implementarse a mano (para obtener el *midstate*) y la minería debe ejecutarse en un *Web Worker* para no bloquear la interfaz.

Id	Sec.	Entradas	Salidas / fórmula
`w-modelos`	1	Modelo: nadie, banco, red de prueba de trabajo	Estado de las dos tiendas (acepta / rechaza)
`w-hash`	4	Entrada A, entrada B	SHA-256 y SHA-256d de cada una; cuadrícula de 256 bits con los distintos; recuento
`w-bloque`	5	Seis campos del encabezado (precargados con `genesis_header.json`)	80 bytes en hexadecimal; SHA-256d invertido; comparación con el objetivo; botón «restaurar génesis»
`w-minero`	6	Dificultad $d \in [1, 28]$	Intentos, intentos esperados 2^d, hashes/s, tiempo; algoritmo con *midstate*
`w-serie`	6	$d \in \{12, 14, 16, 18\}$; bloques $\in \{50, 200, 500\}$	Histograma frente a e^{-x} en múltiplos de 2^d
`w-cadena`	7	Texto de cada uno de 5 bloques; $d = 14$	Estado válido/inválido; «reminar todos»; «restaurar»
`w-dificultad`	8	Escenario; con/sin ajuste	26 periodos; $D_{\text{nueva}} = D \cdot 20160/T_{\text{real}}$ acotado a $[\frac{1}{4}, 4]$; porcentaje de bloques > 30 min (teoría $e^{-3} = 4.98\,\%$)
`w-carrera`	10	$q \in [1, 49]\,\%$; $z \in [1, 20]$; criterio; carreras	Poisson, exacta (binomial negativa) y Monte Carlo con IC de Wilson; gráfica y tabla de confirmaciones
`w-costo`	12	q, horas, USD/kWh	H_a, P_a, electricidad, $P_{\text{éxito}}(z = 6)$ (Resultado 12.1)

A.5 Diseño, formato y accesibilidad

- **Paleta** (variables CSS): azul `#1F4E79`, naranja `#D95F02`, verde `#1B7F5C`, violeta `#6A5ACD`, rojo `#B22222`, gris `#5F6368`. Apta para daltonismo; ningún dato debe depender solo del color (usar marcadores y etiquetas). Debe haber tema claro y oscuro.
- **Tipografía:** serifa para el texto largo, sans para títulos y kickers, monoespaciada para hashes y código. Los hashes largos deben partirse en grupos legibles (8 caracteres) y permitir copiar.
- **Números:** punto decimal y espacio fino como separador de miles (`2 016`, `4 295 032 833`), igual que el documento. Usar escala logarítmica donde el eje abarque órdenes de magnitud y rotularla.
- **Accesibilidad:** cada gráfica lleva su `alt`; los controles deben operarse con teclado; las tablas con `scope`; respetar `prefers-reduced-motion`.
- **Rendimiento:** el minero y las simulaciones de Monte Carlo (hasta 10^6 carreras) deben correr en un *Web Worker* y poder detenerse.

A.6 Vectores de prueba para validar la página

La página convertida debe reproducir exactamente estas cifras (proceden de los datos de este documento y de cálculos independientes):

Prueba	Resultado esperado
SHA-256d del encabezado del génesis (bytes invertidos)	`000000000019d6689c085ae1 65831e934ff763ae46a2a6c172 b3f1b60a8ce26f`; 43 bits iniciales en cero
Intentos esperados con dificultad 1	$4\,295\,032\,833$ ($\approx 2^{32}$)
SHA-256 de «`prueba de trabajo`» y de «`prueba de trabaio`»	`47743df0…18483e` y `cc7cef41…3309fa`; 115 bits distintos
Efecto avalancha, distribución	$\mathrm{Bin}(256, \frac{1}{2})$: media 128, desviación 8
$\mathrm{P}(T > 30\,\mathrm{min})$, $\mathrm{P}(T < 1\,\mathrm{min})$	$e^{-3} = 4.98\,\%$; $1 - e^{-0.1} = 9.52\,\%$
Doble gasto, $q = 10\,\%$, $z = 6$	Poisson $0.024\,3\,\%$; exacta (empate) $0.059\,1\,\%$; exacta (superar) $0.011\,0\,\%$
Doble gasto, $q = 30\,\%$, $z = 6$	Poisson $13.21\,\%$; exacta (empate) $15.65\,\%$; exacta (superar) $8.91\,\%$
Confirmaciones para $< 0.1\,\%$, $q = 30\,\%$	24 (Poisson), 32 (exacta, empate), 29 (exacta, superar)
Confirmaciones para $< 0.1\,\%$, $q = 45\,\%$	340, 539 y 530
Costo, $q = 30\,\%$, 6 h, 0.05 USD h/kW	428.6 EH/s; 6.75 GW; 2.03 MUSD; éxito $z = 6$ (superar) $8.9\,\%$
Bits «cero» exigidos a la red de 2026	$\log_2(10^{21} \times 600) \approx 79$

B Reproducibilidad

Todas las cifras y gráficas de este documento se generan a partir de un único script con semilla fija.

```
pow-latex/
  main.tex                // este documento (XeLaTeX)
  main.pdf
  data/
    genesis_header.json   // 6 campos, 80 bytes en hex, hash, objetivo
    avalanche_hist.csv    // 20 000 pares de SHA-256
    mining_hist.csv       // 600 bloques minados con d = 14
    block_time_dist.csv   // densidad y supervivencia exponencial
    difficulty_scale.csv  // intentos esperados y tiempos segun d
    difficulty_sim.csv    // 4 escenarios x 26 periodos
    double_spend_curves.csv, double_spend_mc.csv, race_paths.csv
    confirmations_table.csv, attack_cost.csv
    energy_mix.csv, halving_schedule.csv, timeline.csv
    summary.json          // cifras agregadas usadas en el texto
  scripts/gen_data.py     // genera todo lo anterior (semilla 20261013)
```

```
python3 scripts/gen_data.py       // ~1 minuto; requiere numpy y scipy
xelatex main.tex                  // dos pasadas
```

El script (a) verifica el hash del génesis con `hashlib`; (b) mide el efecto avalancha con SHA-256 real; (c) mina bloques reales con doble SHA-256; (d) simula el ajuste de dificultad con tiempos gamma; (e) evalúa las fórmulas de Poisson y binomial negativa con `scipy.stats`; y (f) corre el Monte Carlo bloque a bloque con intervalos de Wilson.

C Derivaciones breves

Resultado C.1 — El voto es proporcional al cómputo — Sean m mineros con tasas de hashes $h_1, \dots, h_m$ y probabilidad de éxito p por hash. El tiempo hasta el primer éxito del minero i es, con muy buena aproximación, exponencial de tasa $\lambda_i = h_i p$. Para exponenciales independientes,

$$\mathrm{P}\left(T_i < \min_{j \neq i} T_j\right) = \int_0^\infty \lambda_i e^{-\lambda_i t}\, e^{-(\Lambda-\lambda_i)t}\, dt = \frac{\lambda_i}{\Lambda} = \frac{h_i}{\sum_j h_j}, \qquad \Lambda = \sum_j \lambda_j.$$

Dividir el cómputo entre más identidades no cambia $\sum h_j$ ni la probabilidad de ganar.

Resultado C.2 — Límite exponencial de la búsqueda — Con $p = 2^{-d}$ y $n = x/p$ intentos, $(1-p)^{x/p} \to e^{-x}$ cuando $p \to 0$, con error relativo del orden de p (para $d = 14$, menor que 10^{-4}). La geometría es exactamente sin memoria: $\mathrm{P}(N > n+m \mid N > n) = (1-p)^m$. La mediana satisface $(1-p)^n = \frac{1}{2}$, es decir, $n \approx \ln 2/p$. Si los éxitos llegan a una tasa de hp por segundo (proceso de Poisson), el tiempo entre éxitos es exponencial de media $1/(hp)$: con la dificultad ajustada, 10 minutos.

Resultado C.3 — Ruina del jugador — Sea a_k la probabilidad de que el déficit llegue alguna vez a 0 partiendo de $k \geq 0$, con pasos $+1$ (prob. p) y -1 (prob. q). Por condicionamiento en el primer paso, $a_k = p\, a_{k+1} + q\, a_{k-1}$ con $a_0 = 1$. La ecuación característica $px^2 - x + q = 0$ tiene raíces 1 y q/p, de modo que $a_k = A + B(q/p)^k$. La solución buscada es la mínima no negativa: si $q < p$, $a_k = (q/p)^k$ (pues $a_k \to 0$ cuando $k \to \infty$); si $q \geq p$, $a_k = 1$.

Resultado C.4 — Fórmulas de la sección 9 — Condicionando en $K = k$, el atacante parte de un déficit $z - k$. Para el criterio de empate, el éxito es seguro si $k \geq z$ y ocurre con probabilidad r^{z-k} si $k < z$; para el de superar, hace falta alcanzar el déficit -1, con probabilidad r^{z-k+1} si $k \leq z$ y 1 si $k \geq z+1$. Sumando con los pesos de la binomial negativa se obtienen (2) y (3). Para (1) se sustituye $\mathrm{P}(K = k)$ por el peso de Poisson. Los momentos de la binomial negativa salen de que K es la suma de z variables geométricas independientes, cada una de media q/p y varianza q/p^2.

Glosario

Bits (campo) Notación compacta del objetivo de dificultad dentro del encabezado.

Confirmación Bloque que, a partir del que contiene un pago, se suma a la cadena pública. «z confirmaciones» cuenta el bloque que incluye el pago.

Dificultad Medida de cuánto menor es el objetivo actual respecto al del génesis; dificultad $1 \approx 2^{32}$ hashes.

Doble gasto Usar el mismo activo digital en dos pagos (definición 1.1).

Extranonce Campo adicional dentro de la transacción de recompensa que se varía al agotar el nonce.

Hash criptográfico Función que convierte cualquier entrada en una salida de longitud fija, determinista, con efecto avalancha y resistente a preimagen.

Midstate Estado intermedio de SHA-256 tras procesar los primeros 64 bytes del encabezado.

Nonce Número de 32 bits que el minero varía para cambiar el hash del encabezado.

Objetivo (*target*) Número de 256 bits; un hash válido debe ser menor o igual que él.

Raíz de Merkle Resumen de 32 bytes de todas las transacciones del bloque.

Sybil, ataque Fabricar identidades para ganar influencia.

Referencias

[1] S. Nakamoto, «Bitcoin: A Peer-to-Peer Electronic Cash System», 2008.

[2] D. Chaum, «Blind signatures for untraceable payments», *Advances in Cryptology – CRYPTO '82*, 1983.

[3] D. Chaum, A. Fiat y M. Naor, «Untraceable electronic cash», *CRYPTO '88*, 1990.

[4] S. Haber y W. S. Stornetta, «How to time-stamp a digital document», *Journal of Cryptology*, 1991.

[5] C. Dwork y M. Naor, «Pricing via processing or combatting junk mail», *CRYPTO '92*, 1993.

[6] A. Back, «Hashcash», 1997; «Hashcash – a denial of service counter-measure», 2002.

[7] M. Jakobsson y A. Juels, «Proofs of work and bread pudding protocols», 1999.

[8] W. Dai, «b-money», 1998.

[9] N. Szabo, «Bit Gold», 2005.

[10] H. Finney, «RPOW», 2004.

[11] M. Rosenfeld, «Analysis of hashrate-based double spending», 2012 (arXiv:1402.2009).

[12] C. Grunspan y R. Pérez-Marco, «Double spend races», 2017 (arXiv:1702.02867).

[13] I. Eyal y E. G. Sirer, «Majority is not enough: Bitcoin mining is vulnerable», *Financial Cryptography*, 2014.

[14] Cambridge Centre for Alternative Finance, *Cambridge Digital Mining Industry Report*, abril de 2025.